Food Store - Proyecto integrador

Segunda entrega parcial - Unidades 4 a 8

Exequiel Arancibia / Joaquin Romero · Bases de Datos II · UTN · 10/11/2026

El proyecto conserva el Food Store acumulado y completa su capa de datos con tres motores: PostgreSQL para la operación, MongoDB para una proyección documental y SQLite para el trabajo offline de la tablet. Este informe resume qué se implementó, cómo se probó, qué resultados se observaron y qué propuestas de IA se aceptaron o descartaron. Las fuentes completas y sus salidas acompañan al informe en el repositorio.

Estado técnico: antecedentes y complementos del TPI implementados y probados. La migración FNBC nueva, la tarea diaria y la restauración desde su respaldo terminaron correctamente, con evidencias separadas. La persona solicitante ratificó las conclusiones y decisiones presentadas; la constancia real se conserva en `docs/REVISION_HUMANA.*`.

Una autoridad de negocio, tres formas de almacenamiento

Motor observado	Qué conserva	Responsabilidad
PostgreSQL 17.11	Usuarios, categorías, productos, pedidos, detalles, reglas, índices, FNBC, vistas y permisos.	Única autoridad que valida ventas, precios y stock. Cumple PostgreSQL 16+.
MongoDB 8.0.34	Cinco pedidos existentes con sus siete líneas embebidas, importes históricos y cliente como ID/alias.	Proyección experimental para leer pedidos completos; no decide ventas ni stock.
SQLite 3.53.1	Catálogo acotado, borradores, ítems, solicitudes UUID e instantánea de servidor.	Funciona offline y simula la validación. Un verde no es una venta.

Cobertura de los catorce objetivos

Unidad	Objetivos	Elementos y comprobaciones
4	1, 2 y 3	Dependencias reales, FNBC, migración y reversión probadas después de Kiro, más desnormalización medida.
5	4, 5 y 6	Mínimo privilegio, 44 ACL exactas, auditoría e incidente reconstruido; revisión IA y decisiones justificadas.
6	7 y 8	Respaldo diario disparado por calendario y restauración aislada contra reloj: RTO nuevo 13,4303715 s; salud, mantenimiento y post-incidente.
7	9 y 10	Documentos MongoDB, consultas verificadas y comparación funcional con PostgreSQL; elección de autoridad relacional.
8 / TPI	11, 12, 13 y 14	SQLite con Kiro/OpenCode, conflictos, dos laboratorios, review acumulado, Git, mantenimiento, bitácora y DUIA. Ratificación humana registrada.

La base fuente conservada tiene 2 categorías, 20.001 usuarios, 50.000 productos, 225.000 pedidos y 499.997 detalles. No se adjuntan sus filas personales ni respaldos privados. Se integraron 456 archivos históricos con sus bytes y hashes originales; los complementos del TPI se identifican aparte, con fecha y procedencia propias.

Lectura de evidencias: cada resultado corresponde al laboratorio fechado de su unidad. Integrar archivos no equivale a reejecutar esas pruebas. Las rutas de este informe son relativas a la carpeta food-store-integrador.

Unidad 4 - Esquema y redundancia

Dependencias reales y FNBC

Se refactorizó la relación de control de lotes `ControlLoteAlmacen(L,D,R)`: lote, depósito y responsable. Las dependencias verificadas son **LD -> R** y **R -> D**; las claves candidatas son LD y LR. Los tres atributos son primos, pero R no es superclave: R -> D viola FNBC. La afirmación corresponde a esta extensión del esquema, no certifica por sí sola todas las relaciones heredadas.

Decisión	Fundamento
Descomponer en RD y LR	`responsable_deposito(R,D)` tiene PK R; `lote_responsable(L,R)` tiene PK LR. Ambas relaciones están en FNBC.
Reunión sin pérdida	La intersección es R, que determina RD. El JOIN reconstruye las tres filas originales; EXCEPT ALL en ambos sentidos da cero diferencias.
Conservar LD -> R	Las PK/FK locales no preservan esta dependencia. Ambas tablas serializan escrituras mediante la misma guardia; se valida su JOIN. Dos sesiones reales mostraron espera Lock y rechazo 23505 de la colisión.

Migración y plan de reversión

La migración histórica U4 usa BEGIN/COMMIT, valida R -> D, conserva la tabla original renombrada, proyecta RD/LR y crea una vista compatible. Se comprobaron tres filas reconstruidas, diferencias bilaterales cero y conservación de las huellas históricas. El respaldo anterior se identifica por SHA256; el dump permanece privado.

Complemento nuevo TPI, 10/11/2026: Kiro produjo tres specs antes del código (commit 89f8102); Build y revisión final precedieron al ensayo (fuentes c3711cc). En una copia aislada, la FD inválida rechazó el mismo migrar.sql con 23514. ROLLBACK y COMMIT conservaron tres filas y cero diferencias; la concurrencia observó Lock, confirmó A y rechazó B con 23505. Después se confirmaron cambios válidos que producen cuatro filas. El down con error P7501 restauró por rollback el catálogo y esas cuatro filas; el down confirmado reconstruyó las cuatro desde el JOIN actual, conservando el checkpoint original de tres. Hubo once controles de fase y SUCCESS en 139,056 s, con cinco huellas centrales y otra de extensiones intactas en copia/fuente. No se reatribuye a Kiro la migración histórica U4.

Desnormalización controlada y costo observado

La materializada diaria de categorías evita recalcular el agregado de lectura. Conserva la autoridad de las tablas fuente y es independiente de la materializada mensual heredada. Adapta TIMESTAMPTZ al día local semiabierto y respeta subtotal generado y PK compuesta; no inventa una columna de baja en detalle.

Medición histórica del 20/10/2026	Resultado
Consulta base, mediana	173,044 ms
Consulta a la materializada, mediana	0,095 ms
Refresco de la materializada	922,234 ms
UPDATE con su mantenimiento	930,009 ms

Son cinco corridas con caché caliente más un calentamiento; los resultados coinciden. La lectura mejora en ese ensayo a costa de refresco y escritura, sin promesa universal. Hay instalación transaccional, bloqueo asesor, READ COMMITTED obligatorio, refresco dentro de la escritura y rechazo de TRUNCATE. Veinte escenarios y concurrencia terminaron con cero diferencias y cero totales incorrectos. La reversión de desnormalización retira sólo esa caché, no sustituye el down FNBC nuevo.

Fuentes: unidad4/tp_fnbc_control_lote.sql y evidencias/; complementos/migracion_fnbc/; docs/kiro_migracion/; docs/evidencias_tpi/migracion_fnbc.json; unidad4/tp_desnormalizacion_top_categorias.sql y reversion_desnormalizacion.sql.

Unidad 5 - Administración y seguridad

Roles y permisos con mínimo privilegio

Se conservaron seis roles NOLOGIN y dos LOGIN. Ninguno tiene SUPERUSER, CREATEDB, CREATEROLE, REPLICATION ni BYPASSRLS. La matriz efectiva coincide con **44 ACL esperadas, cero faltantes y cero adicionales**. Hay exactamente tres membresías: INHERIT verdadero, SET falso y ADMIN falso.

Grupo de roles	Decisión y protección
rol_propietario / rol_funciones	Separar propiedad de objetos y ejecución privilegiada. Cuatro funciones SECURITY DEFINER tienen dueño limitado, search_path fijo y referencias calificadas.
rol_app_lectura / rol_app_escritura	Permisos acotados a la operación. El usuario se expone por columnas; la app no recibe hashes, escritura libre de total ni administración del catálogo.
rol_soporte / rol_reportes	Accesos puntuales. Se descartó heredar lectura amplia para soporte y SELECT general sobre public para reportes.
app_web / admin_datos	La app hereda lectura/escritura; administración hereda soporte. No son propietarios ni superusuarios.

Se observaron 52 eventos de permisos y 71 de objetos futuros aprobados, con conexiones reales y pruebas revertidas. Las ACL por columna no son RLS ni aislamiento por fila. Los permisos existentes ajenos al modelo no se presentan como saneados universalmente.

Auditoría e incidente reconstruido

Se activó auditoría nativa de conexiones, desconexiones y sentencias, con jsonlog/stderr y límites de parámetros en cero. Registra intentos incluso si terminan en rollback; no es un historial de cada valor OLD/NEW ni prueba de COMMIT. Un literal sensible escrito en el SQL todavía podría aparecer: la confidencialidad también depende del cliente.

El ensayo B dejó 14 eventos e incluyó login fallido 28P01 y operaciones de prueba revertidas. El simulacro C conservó **17 eventos**: tres autenticaciones negativas y una positiva de negocio; SELECT autorizado y un intento de `GRANT admin_datos TO app_web` denegado con **42501**. El cliente recibió y descartó 20.001 filas; la membresía administrativa era falsa antes y después.

OpenCode recibió únicamente el log anonimizado, sin guion del cliente ni sus conteos. Su reconstrucción se contrastó con la ejecución. Una lectura autorizada no demuestra por sí sola exfiltración, compromiso ni identidad personal del actor.

Decisiones de la auditoría asistida

Se aceptaron con alcance tres riesgos: secretos literales en logs, ACL ajenas preexistentes y canal temporal/falta de limitación de intentos. Se rechazaron las dos alternativas amplias porque contradicen el mínimo privilegio. Ante un incidente real confirmado: preservar evidencia, reconstruir el alcance y evaluar restringir la cuenta con intervención responsable. **No se ejecutó contención automáticamente.**

La conformidad histórica U5 se conserva con su alcance real; no acredita lectura exhaustiva por ambos alumnos. La ratificación posterior de las decisiones del integrador se conserva separadamente en `docs/REVISION_HUMANA.*`, con los límites allí indicados.

Fuentes: unidad5/sql/roles.sql; unidad5/docs/evidencias/a_comparacion_exacta.json; permissions_resultados.json y defaults_resultados.json; unidad5/docs/log_simulacro_anonimizado.txt; unidad5/docs/informe_incidente.md; unidad5/docs/ia/a_revision_validacion.md.

Unidad 6 - Continuidad y recuperación

Plan, automatización y tareas rutinarias

El plan U6 fue especificado antes de consultar OpenCode. Codex integró y corrigió su propuesta de backup/restauración, el helper y los SQL finales; no se atribuye todo el código literal a OpenCode. Los scripts comprueban códigos de salida y abortan ante estados inesperados.

Tarea	Frecuencia propuesta	Control y evidencia
Backup custom	Diario 02:00	pg_dump -Fc, archivo no vacío, TOC legible, SHA256 y manifiesto SUCCESS sin sobrescribir.
Salud	Diario 02:15	Tamaño, conexiones, tablas y consultas activas; ON_ERROR_STOP y salida registrada.
Mantenimiento	Domingo 03:00	Autovacuum/estadísticas y tuplas muertas; ANALYZE puntual justificado. Sin VACUUM FULL periódico.
Restore de prueba	Domingo 03:30	Nueva base aislada, validaciones y huellas. Si el destino existe, aborta sin reemplazarlo.

Los horarios se interpretan en UTC-03. U6 ensayó las tareas manualmente. El **complemento TPI** registró una tarea DIARIA real con primer disparo de ensayo a las 14:37:41; el calendario, sin inicio manual, produjo backup SUCCESS y LastTaskResult=0. Usó token interactivo, nivel limitado y ninguna contraseña guardada. Después se retiró únicamente esa tarea: acredita configuración/ejecución programada de laboratorio, no un servicio permanente activo.

Simulacro de desastre e informe post-incidente

El 03/11/2026 se restauró el último respaldo SUCCESS de la copia `foodstore` en `foodstore_restore_test`. Se conservaron la fuente y el origen U5. El cronómetro monotónico comenzó antes de createdb y terminó después de aislamiento, restauración y validaciones, sin incluir las comprobaciones posteriores.

Medida	U6 - 03/11/2026	TPI - 10/11/2026
RTO medido	11,3500773 s	13,4303715 s
Antigüedad al iniciar	31,8083982 s	19,0739065 s
Respaldo validado	Custom; 8.224.005 bytes	Custom; 8.224.005 bytes
Conteos y huellas	Cinco tablas; iguales	Cinco tablas; iguales
Productos/detalles inválidos	0 / 0	0 / 0

El objetivo de laboratorio era RTO <=120 s y RPO <=24 h con backup diario exitoso. La antigüedad usa el inicio de pg_dump como referencia conservadora; no es pérdida observada ni PITR. Fuera del RTO, el postflight verificó 12 tablas base, 22 relaciones, 10 funciones, 25 índices, 35 restricciones, 15 triggers y ambas materializadas.

El post-incidente registra la corrección de variables libpq, un fallo de pg_dump controlado que no publicó copia válida y el rechazo de restaurar sobre un destino existente. Propone una segunda copia cifrada externa. La restauración usó **--no-owner y --no-privileges**: recupera datos/objetos en aislamiento, pero no demuestra recuperar la seguridad U5. No se habilitó la copia a la aplicación.

Post-incidente TPI: el 10/11 el script U6 intacto restauró el mismo último backup programado, con hash verificado. La copia anterior se conservó por RENAME y se creó un destino nuevo; coincidieron cinco conteos/huellas. Se corrigieron antes del éxito la codificación XML/COM y un verificador que exigía un nodo omitido por Windows: se conservó la omisión y se verificó Limited mediante CIM. Los intentos fallidos se archivaron, sin inventar respaldos. Para continuidad operativa se requiere gestionar ventanas, monitoreo y copia cifrada externa.

Fuentes: unidad6/SPEC_U6.md, scripts y docs/informe_post_incidente.md; complementos/mantenimiento_programado/evidencias/scheduler_ejecucion.json y restauracion_tpi_resultado.json; complementos/mantenimiento_programado/informe_post_incidente_tpi.md.

Unidad 7 - Modelo documental

Subconjunto real del proyecto y tipos exactos

Se seleccionaron los pedidos existentes **1 a 5** del Food Store acumulado, sin agregar pedidos a la fuente. La exportación PostgreSQL usa REPEATABLE READ READ ONLY. MongoDB conserva cinco documentos y siete líneas embebidas; los clientes viajan sólo como ID y alias sintético. Se excluyen nombres personales, correos, teléfonos y hashes.

Elemento	Modelado y decisión
Identidad e información del pedido	_id Long; fecha UTC; estado, forma de pago y total Decimal128. Se conserva la identidad del servidor.
Líneas embebidas	Producto Long, cantidad Int32, precio histórico y subtotal Decimal128. El pedido se recupera completo desde una colección.
Cliente	ID de usuario y alias de presentación; copia al exportar, sin sincronización automática de perfiles.
Integridad	JSON Schema más reglas de coherencia matemática y producto único por pedido; índices de estado y multikey por producto.

Las cinco fechas fueron verificadas sin fracciones submilisegundo, evitando atribuir pérdida de precisión a esta exportación. Los importes históricos no se reemplazan por el precio actual del producto.

Consultas y resultados verificados

Consulta	MongoDB	Contraste realizado
Todos los pedidos	5 documentos, 7 líneas	Contenido igual a la exportación PostgreSQL.
Estado ENTREGADO	IDs [1,2,3,4]	PostgreSQL: mismos cuatro IDs.
Contiene producto 1	IDs [1,3,5]	PostgreSQL EXISTS: mismos tres IDs.
Total >3000	IDs [3,4]	Consulta Mongo adicional; no se presenta un contraste SQL nuevo.

Ambos clientes finalizaron con código cero. Las pruebas negativas rechazaron total incoherente, cantidad cero y producto repetido con código **121**, y el ID duplicado con **11000**. Al cierre seguían cinco documentos y cero inválidos persistidos. Las cinco huellas de la fuente PostgreSQL permanecieron iguales.

Comparación práctica y elección de tecnología

La estructura documental evita unir colecciones para recuperar un pedido con sus líneas. Eso no prueba mayor velocidad: no se hizo benchmark. Embeber tampoco garantiza la suma correcta; los validadores explícitos rechazaron los casos incoherentes. Se contrastaron las dos afirmaciones de IA con la documentación oficial y el laboratorio, sin afirmar que MongoDB carezca de transacciones.

Se eligió **PostgreSQL como fuente principal**: conserva las relaciones, reglas y permisos ya implementados, y decide precio, stock y venta. MongoDB se mantiene como proyección experimental de lectura, útil si existe una necesidad concreta y se define su actualización. Cinco pedidos permiten verificar migración y consultas, no justificar sustituir toda la capa transaccional.

Fuentes: unidad7/consultas_mongodb.js; consultas_postgresql.sql; pedidos_migrados.ejson; docs/evidencias/validacion.json y mongodb_salida.jsonl; docs/evidencias/precision_y_dominios.json; docs/ia/consulta_respuesta.md.

Unidad 8 - Tablet y sincronización

Diseño, generación y reglas locales

Kiro produjo requirements, design y tasks reales antes de OpenCode Build. Codex conservó las respuestas y extrajo los bloques; no se atribuye escritura automática a Kiro. Build generó seis fuentes; las correcciones y SQL30 posterior se identifican por separado. Se preservan originales, prompts, metadatos y Git.

SQLite usa cuatro tablas STRICT y una auxiliar de instantánea. Cada conexión activa y verifica foreign_keys. Los importes son centavos INTEGER64; PostgreSQL convierte NUMERIC a BIGINT. `solicitud_uuid` es PK e `id_local` referencia al borrador. Un índice parcial permite una sola solicitud PENDIENTE por borrador y conserva intentos históricos.

El CSV procede de una exportación real psql: disponibilidad derivada de stock>0 y baja de producto o categoría. El importador valida t/f y hace UPSERT, sin borrar referencias. El semáforo compara con la instantánea nueva, usa LEFT JOIN y da prioridad al rojo sobre el cambio de precio. Sólo actualiza solicitudes PENDIENTE.

Casos obligatorios y resolución demostrada

Caso	Preparación	Resultado observado
C1	Operación offline y catálogo local	COMMIT conserva borrador completo; consulta sin comandos PostgreSQL. ROLLBACK alternativo deja cero filas.
C2	Muzzarella 9000 ->9100	CONFLICTO; aceptación sintética, precio actualizado y nuevo UUID -> SIMULADO_APROBADO. El anterior conserva el conflicto.
C3	Cantidad 5, stock 4	RECHAZADO por stock insuficiente; no se descuenta stock.
C4	Baja lógica de Agua	RECHAZADO; lápida conservada y producto oculto del catálogo visible.
C5	Precio/stock coherentes	SIMULADO_APROBADO; no se crea pedido ni reserva en servidor.
C6	UUID repetido y reejecución	PRIMARYKEY rechaza duplicado; reejecutar no cambia estado, motivo ni fecha de históricos.
C7	Cantidad cero y producto inexistente	CHECK y FOREIGNKEY reales; cada ROLLBACK retira también el encabezado previo.

Pasaron además siete bordes: segundo pendiente, mezcla rojo/precio, borrador vacío, ausente, indisponible, stock cero y rojo/precio en el mismo ítem. Un error de sentencia no revierte por sí solo toda la transacción: se ejecutó rollback explícito.

Dos laboratorios, conservación y alcance

Dos corridas desde copia PostgreSQL nueva y SQLite vacía dieron la misma semántica: **9 solicitudes = 2 CONFLICTO +4 RECHAZADO +3 SIMULADO_APROBADO**. integrity_check fue ok y foreign_key_check no devolvió filas. La primera copia se conservó renombrada; la segunda quedó como copia canónica. La SQLite entregada es la primera; UUID, horas y bytes pueden diferir entre corridas.

La copia conservó 225.000 pedidos, stock global 5.000.038 y las huellas de pedidos/detalles/stock. La fuente conservó cinco huellas completas. Sólo hubo preparaciones administrativas C2/C4 dentro de la copia. No se implementa una venta productiva: UUID definitivo y creación de venta tendrían que validarse en la misma transacción del servidor.

Fuentes: unidad8/movil/; tools/reproducir.py; docs/kiro/; docs/evidencias/reproduccion_u8.json; resumen_verificaciones.json; segunda_ejecucion_comparacion.json; segunda_ejecucion/.

Revisión final y uso de IA

Cobertura completa y decisiones trazables

OpenCode Plan revisó **83 fuentes anteriores y siete fuentes finales U8**, con adjuntos divididos y continuaciones para no omitir líneas. Se conservaron 17 consultas de antecedentes y los reviews U8 previo/final. La matriz final reúne **43 grupos y 64 referencias originales**, sin decisión técnica pendiente: 32 grupos aceptados, siete descartados y cuatro corregidos. Por referencias son 49/9/6, respectivamente.

Los cuatro grupos corregidos son RH04, U801, U802 y U803: guarda de estado inicial; protección privada explícita del runtime; huellas completas de conservación; eliminación de un conflicto histórico artificial. Los riesgos condicionados aceptados no se presentan como incidentes ocurridos o correcciones ejecutadas.

El TPI añade cinco decisiones de seguridad y catorce de los complementos (SCH01-SCH07 y FD01-FD07). Los reviews finales cubren cuatro fuentes del programador y seis de migración; el runner largo fue recibido en dos partes contiguas. FD07 corrigió antes de ejecutar una regex que rechazaba dígitos en nombres internos de SAVEPOINT. No se alteraron las respuestas originales.

Grupo	Contraste desarrollado en PostgreSQL	Decisión y alcance
RH01	app_web intentó borrar el único detalle de un pedido final: rechazo 23514.	Descartado como defecto actual: la guarda heredada ya protege el acumulado.
RH02	UPDATE total como app_web: rechazo 42501, privilegio de columna ausente.	Aceptado como límite estructural de un escritor privilegiado; no vulnerabilidad demostrada de la app actual.
RH04	Administrador insertaba CONFIRMADO vacío; SQL30 lo rechaza 23514 y permite PENDIENTE.	Corregido sólo en copias U8; conserva transiciones anteriores. Probado también en la segunda copia.

Todas estas pruebas terminaron en ROLLBACK y cinco huellas de tablas iguales. La identidad de la copia puede avanzar pese al rollback; no se afirma conservación de secuencias ni de todo el clúster. La matriz original conserva por separado hipótesis, pruebas efectivas y decisiones.

Bitácora y Declaración de Uso de IA

Herramienta	Finalidad, material y decisión
Kiro	Specs genuinas de roles U5, SQLite U8 y migración nueva TPI, previas al código. Se conservan prompts, respuestas y extracción literal; el fragmento incompleto original no se ocultó. Aclaraciones Codex separadas, sin reatribuir la U4 histórica.
OpenCode	Seguridad/log U5, mantenimiento U6, Build/Plan U8 y complementos TPI. Build FNBC generó cinco archivos; el helper posterior es Codex y se revisó entre seis fuentes. Se conservan metadatos y snapshots; Plan no ejecutó SQL.
Codex	Integración, lectura técnica, correcciones, ejecución local, conservación de evidencia, revisión independiente y preparación del informe. Se declara como asistencia automatizada, sin suplantar firmas de alumnos.

No se enviaron a las IA dumps, filas personales, contraseñas ni logs crudos privados; se usaron contexto y copias saneadas autorizados. Los secretos se mantuvieron fuera de Git. Los commits técnicos no se atribuyen a los integrantes. La bitácora U8 contiene ocho entradas y el TPI conserva las de cada unidad.

Ratificación humana registrada: después de presentar el informe y la matriz, la persona solicitante respondió: **"Sí, revisé y ratifico las conclusiones y todas las decisiones propuestas"**. Abarca los 43 grupos/64 referencias, cinco decisiones de seguridad y catorce de complementos. `docs/REVISION_HUMANA.*` conserva la respuesta y los hashes de lo revisado; `docs/DECISIONES_REVIEW.*` mantiene los fundamentos ratificados. La constancia no es una firma ni certifica lectura línea por línea o defensa de ambos integrantes; tampoco reatribuye la revisión automatizada previa.

Fuentes: unidad8/docs/review_hallazgos.*; review_fuentes_manifest.json; docs/ia/; bitacora_ia.md; DUIA.md; evidencias/review_bd_*.json. Los complementos nuevos requieren su propia revisión previa a la ejecución.

Verificación, reproducción y defensa

Índice de los catorce objetivos

N.º	Evidencia principal dentro del paquete	Informe
1	unidad4/tp_fnbc_control_lote.sql y evidencias/fnbc_pruebas.json	p. 2
2	complementos/migracion_fnbc/; docs/kiro_migracion/; docs/evidencias_tpi/migracion_fnbc.json	p. 2
3	unidad4/tp_desnormalizacion_top_categorias.sql, métricas y reversion_desnormalizacion.sql	p. 2
4	unidad5/sql/roles.sql; docs/evidencias/a_comparacion_exacta.json	p. 3
5	unidad5/docs/log_simulacro_anonimizado.txt e informe_incidente.md	p. 3
6	unidad5/docs/ia/a_revision_*; docs/REVISION_HUMANA.* y DECISIONES_REVIEW.*	pp. 3 y 7
7	unidad6/SPEC_U6.md y scripts; complementos/mantenimiento_programado/evidencias/scheduler_ejecucion.json	p. 4
8	complementos/mantenimiento_programado/evidencias/restauracion_tpi_resultado.json y su post-incidente	p. 4
9	unidad7/consultas_mongodb.js, pedidos_migrados.ejson y evidencias	p. 5
10	unidad7/docs/evidencias/validacion.json y docs/ia/consulta_respuesta.md	p. 5
11	unidad8/docs/kiro/, docs/ia/generacion_* y movil/schema_sqlite.sql	p. 6
12	unidad8/docs/evidencias/reproduccion_u8.json y segunda_ejecucion_comparacion.json	p. 6
13	unidad8/docs/review_hallazgos.*; docs/DECISIONES_REVIEW.* y REVISION_HUMANA.*	p. 7
14	Repositorio Git único, fuentes U4-U8, specs, planes, bitácoras, DUIA y guion	pp. 1, 7 y 8

Reproducción responsable

Se requiere el Food Store acumulado U1-U3 y sus checkpoints posteriores, una copia local y un respaldo privado del operador. Los dumps se excluyen por privacidad; `PROCEDENCIA_UNIDADES.json` acredita las copias exactas de los antecedentes. El DDL sin filas conservado como contexto no reconstruye por sí solo los datos/roles históricos. Cada README indica su checkpoint, conexión y orden; no aplicar toda la historia sobre la base final.

Para repetir U7 se usa sólo lectura sobre la fuente y MongoDB aislada. Para U8 se copian sólo las siete fuentes a otra carpeta y se crea una copia PostgreSQL nueva; no se sobrescriben la SQLite o evidencias entregadas. Los generadores y empaquetadores de las unidades se conservan como antecedentes; el TPI se versiona y empaqueta por separado. Los nuevos complementos quedan identificados y no alteran las fuentes históricas.

Mantenimiento integrado y defensa de diez minutos

Tras restaurar un PostgreSQL viejo, todas las tablets invalidan cachés e instantáneas y descargan catálogo completo, incluidas lápidas. Se conservan borradores/UUID y se reconcilian resultados con el servidor antes de reenviar. No decide la hora del teléfono ni se reutiliza un verde anterior. El encargado revisa conflictos al reconectar, cada quince minutos y al cierre; soporte trata los pendientes diariamente.

La defensa propuesta distribuye diez minutos: 2 de arquitectura/FNBC; 2 de permisos e incidente; 2 de recuperación con RTO; 2 de MongoDB/SQLite y conflicto; 2 de review, decisiones y conservación. El equipo debe poder explicar los scripts, el rollback y los límites de cada prueba. El guion no acredita una defensa ni ensayo ya realizados. Las pruebas técnicas están completas y la persona solicitante ratificó las conclusiones y decisiones, según la constancia conservada.

Referencias: consigna de la segunda entrega y materiales de la cátedra U4-U8; informes y fuentes originales incluidos. Procedencia detallada, prompts y respuestas acompañan a los scripts. No se sustituye evidencia por una afirmación de IA.